Observing Linux calls and implementing xv6 trace

Tacu Igor, FAF 242
Laboratory Works 9 and 10, Operating Systems
Revised implementation and reproduction on 9 November 2026

What can be checked in this submission

Lab 9 observes calls made by a Linux executable, including the descriptor used for terminal output. Lab 10 adds an observation facility inside xv6. The two tasks meet at the system-call boundary, but use different mechanisms: Linux strace is an external tracing program, whereas the xv6 handler and dispatcher are modified kernel code.

The source is now in the same `OS` repository as this report. `Lab9-10/syscalls.c` is the Linux utility. The full `xv6` directory contains `user/trace.c`, the assembly-stub generator, the handler, the syscall number and dispatch entry, the per-process mask, and inheritance in process creation. A reader does not need to locate a separate fork to assess the implementation. Upstream MIT code and the changes made for these labs are distinguished in `xv6/PROVENANCE.md`.

The new evidence was collected in the Ubuntu 24.04 x86_64 server and from QEMU booting that source tree. Console blocks in this report are excerpts of real text recordings, not reconstructed SSH screenshots. The complete Linux traces and xv6 console output are included under `Lab9-10/evidence`; the runner scripts are committed alongside them.

Lab 9 Finding the output descriptor of ls

I compiled the program in a temporary working directory and traced a plain `ls -l` there using `strace -o ls.trace -e trace=openat,read,write,close ls -l`. Sending the trace to its own file keeps strace diagnostics separate from the command's normal stdout. The recorded output call is:

```
write(1, "total 20\n-rw-r--r-- 1 root root "..., 111) = 111
```

Descriptor 1 is standard output. The return value equals the requested byte count in this call, showing that the full buffer was accepted. The buffer includes the long-format listing rather than a message from strace. Its total length depends on the current directory and listing layout, so the byte count is not a portable constant. The implementation of ls can buffer multiple displayed lines before making one write; one displayed line should not be assumed to imply one system call.

The trace also contains file operations unrelated to the file the lab program will create: the dynamic loader and library setup open and read their own files. This matters when interpreting raw strace output. A read on descriptor 3 near process startup does not necessarily read `note.txt`; identifying the pathname at open time is needed to attribute later uses of that descriptor. The complete `ls.trace` is retained so these setup calls can be inspected without reducing the trace to a selected screenshot.

Counting the selected lines in that recording gives 38 calls: 10 openat, 16 close, 11 read, and 1 write. This is the total for the four traced families; calls excluded by the filter cannot be counted from this recording.

Lab 9 Six calls on note.txt

The utility creates or truncates `note.txt` with mode 0644, writes `hello, kernel` plus a newline, closes it, reopens it read-only, reads into a 64-byte buffer with space reserved for the terminator, and closes it again. Both opens are checked before their descriptors are used. A failed read is checked before indexing the buffer, so a negative return value cannot become an invalid array offset. Every close is also checked.

These are the six consecutive file calls captured in the new run:

```
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(3, "hello, kernel\n", 14)         = 14
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
read(3, "hello, kernel\n", 63)          = 14
close(3)                                = 0
```

There are thirteen visible characters plus the newline, hence fourteen bytes. The second read requests at most 63 bytes and receives fourteen. Adding a null byte at `buf[n]` makes the returned bytes suitable for a subsequent string print without writing beyond the buffer. That terminator is a userspace operation and does not appear as another file write.

The repeated return value 3 is descriptor reuse, not a second permanent identity for the file. Descriptors 0, 1, and 2 were already occupied by standard input, output, and error. Closing the first descriptor made slot 3 available again, so the next open could choose it. The same integer could name a completely different open file in another part of the process. The path, inode, and process-local descriptor are related concepts but cannot be substituted for one another.

Although the source calls C `open`, this libc used the Linux `openat` syscall with `AT_FDCWD`. The path is relative to the current working directory. This is a concrete example of why a C function's name and the traced syscall name need not be identical: the library selects the actual interface used to request the kernel operation.

Lab 9 The program also writes to stdout

The status messages use printf, while file data uses write directly. In the recorded run the program's stdout was redirected to the evidence log. The stdio messages were buffered and appeared together in one eighty-byte write:

```
write(1, "open returned descriptor 3\nwrite sent 14 bytes\nread got 14 bytes: hello, kernel\
    n", 80) = 80
```

That explains why the status text can appear late in the syscall trace even though the corresponding printf statements occur between file operations in the C source. Buffering changes when bytes are handed to the kernel. It does not mean the source statements executed in a different order. This stdout call is distinct from `write(3, ...)`, which wrote the fourteen file-data bytes. Keeping both calls in the report makes the descriptor distinction directly visible.

Lab 9 Counting one kind of call

The repeated count used `strace -c -e trace=openat ls -l` with stdout redirected to `/dev/null`. The observed summary was:

```
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
100.00    0.000125          12        10           openat
------ ----------- ----------- --------- --------- ----------------
100.00    0.000125          12        10           total
```

This run counted ten openat calls without errors. The earlier remote Ubuntu run counted 31 calls and 18 failed lookups. Both are observations of their respective environments: the new run fixes `LC_ALL=C` and uses the C locale, while the earlier run used a different locale configuration. Failed locale-candidate opens need not make ls fail. It would be incorrect to present 31 as the number every ls invocation must make, or to merge a count from one environment with a trace from another.

The brief lists a counting part without a detailed procedure. The summary command above makes the chosen method explicit. Its timing columns measure this particular instrumented execution; they are not a benchmark comparison, and the single short run does not justify conclusions about system-call performance.

Lab 10 Following number 23 across the boundary

The implementation reserves 23 as `SYS_trace` in `kernel/syscall.h`, after the existing sync call at 22. `user/user.h` declares `trace(int)`, and `user/usys.pl` emits its userspace assembly wrapper. The wrapper places the syscall number in a7 and executes ecall; the mask travels as the first argument. `user/trace.c` validates that a command was supplied, calls trace with the parsed mask, then invokes exec on the command and its remaining arguments.

The kernel counterpart is `sys_trace` in `kernel/sysproc.c`. It retrieves argument zero with argint and saves it in the current process. Returning zero tells the userspace utility that enabling the mask succeeded. Adding the handler to the dispatch array and declaring its prototype makes the number executable rather than merely known to the header. Without that registration, the intermediate experiment reported `unknown sys call 23`; the final tree contains the actual handler and dispatch entry.

The mask is stored in `struct proc`, initialized to zero when allocating a process, and copied from parent to child during kfork. Exec keeps the process while replacing its program image, so it also keeps this process field. A new independently allocated process starts with no tracing. The reset on allocation avoids retaining a previous occupant's mask when a process-table slot is reused.

Lab 10 Check 1 and patch validation

The submitted `trace.patch` is a Git diff against MIT xv6 commit `e93dabeb98431235c6e1a101cb96c73c923fdbc4`. It must be applied to that base, which already has sync at number 22. Other xv6 versions may use different source contexts or syscall numbers. The full integrated tree is also included in this repository.

A fresh disposable checkout was used to run `git apply --check`, apply the patch, and compile and boot it. To reproduce Check 1 explicitly, the test temporarily omitted only `[SYS_trace] = sys_trace` from the dispatch table, keeping the user wrapper present. This intentionally incomplete control produced:

```
trace 32 grep hello README
3 trace: unknown sys call 23
trace: failed
$
```

The test then restored the dispatch entry and rebuilt. Read, open and close masks worked, and `trace 2 usertests forkforkfork` ended with `ALL TESTS PASSED`. The test verifies that the mask field belongs inside `struct proc` through a successful kernel build, and exercises inheritance through real child processes. The complete build and boot record is `evidence/trace-patch-validation.txt`. The reproducible checker is `scripts/check-trace-patch.py`; use it only with a clean disposable checkout because it applies the patch and rebuilds the generated disk.

Lab 10 Reporting the completed call

The dispatcher first executes `syscalls[num]()` and stores the result in the saved a0 register. It then tests the selected bit and prints the PID, a human-readable name, and the return value through printk. Printing after the call is essential: printing before dispatch would observe the argument or old register contents rather than the completed return value. The name array uses designated initializers corresponding to the header constants, rather than relying on an assumed contiguous textual order.

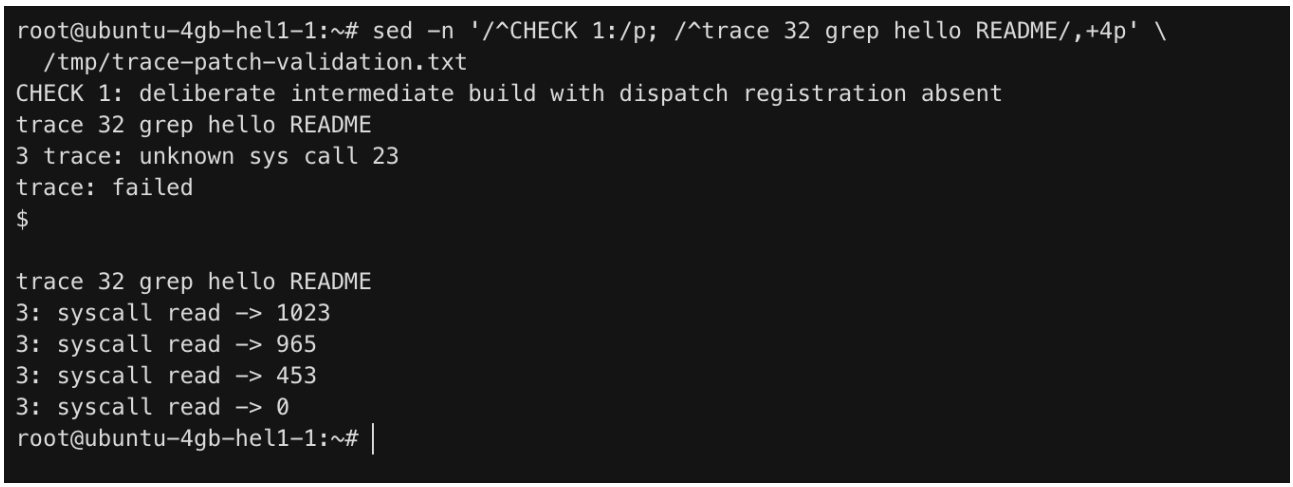


Figure 1: Native Terminal screenshot supplied by the author: saved audit output showing the intentionally unregistered syscall and the restored implementation. The sed command displays both matching runs from the audit log.

The condition also checks that num is below 32 before shifting a 32-bit mask. The implemented call numbers are below that limit. This protects the shift operation but does not turn the lab into a general tracer for arbitrary larger syscall-number spaces. Calls that never return, such as exit, do not pass through this post-return print site. The report therefore does not claim that the all-bits mask must print a line for every possible process event.

For mask 32, bit five selects SYS_read. The observed command produced:

```
$ trace 32 grep hello README
95: syscall read -> 1023
95: syscall read -> 965
95: syscall read -> 453
95: syscall read -> 0
```

The three positive results sum to 2441, the README's length in this image, followed by a zero-length read at EOF. PID values identify this run and may change with the number of preceding shell commands. The mask explains the absence of open and close lines; it does not mean grep stopped using those calls.

Lab 10 Masks that expose different behavior

An all-bits-positive mask of 2147483647 reported trace itself, exec, open, the read sequence, and close. Trace can report its own successful return because the handler installs the mask before the dispatcher's post-call test. Exec returned 3 here, the argument count used by this xv6 implementation, rather than the zero that someone might expect from a different interface.

For write-only tracing, 65536 sets bit sixteen. Echo wrote two bytes for `hi` and one for the newline; the trace message appeared between those operations. For open and close only, the mask is `(1 << 15) | (1 << 21) = 2129920`. The observed output contains precisely those names:

```
$ trace 2147483647 grep hello README
96: syscall trace -> 0
96: syscall exec -> 3
96: syscall open -> 3
96: syscall read -> 1023
96: syscall read -> 965
96: syscall read -> 453
96: syscall read -> 0
96: syscall close -> 0
$ trace 65536 echo hi
hi97: syscall write -> 2

97: syscall write -> 1
$ trace 2129920 grep hello README
98: syscall open -> 3
98: syscall close -> 0
```

The fork-only mask is 2, selecting bit one. `usertests forkforkfork` created enough children to reach the process-table limit, so a return value of minus one was expected at saturation. Multiple PIDs printing lines demonstrate the inherited mask rather than one parent printing all events. The final test result was:

```
102: syscall fork -> 148
103: syscall fork -> 156
103: syscall fork -> 157
103: syscall fork -> 158
103: syscall fork -> 159
103: syscall fork -> 160
103: syscall fork -> -1
OK
ALL TESTS PASSED
$ echo tracing-off
tracing-off
$
```

The subsequent ordinary `echo tracing-off` produced its text without a syscall trace. This checks the independent-process case as well as inheritance. The test runner fails if the distinctive read, write, open, saturation, and success markers are absent, and retains the whole console stream rather than only the final success line.

Limits and conclusion

The Linux and xv6 observations demonstrate two ways to inspect a system-call interface. The Linux trace includes loader and library work outside the explicit C statements. The xv6 facility instead sits directly in the teaching kernel's dispatcher and is intentionally limited to a per-process bit mask, a name, and a return value. It does not decode pointers or copy user buffers like a general strace implementation.

The additional native L9 screenshots use direct terminal stdout: the program emits three writes of 27, 20 and 33 bytes (80 in total), whereas the earlier piped recording combined them into one buffered write. The six calls on fd 3 are unchanged. The first fixed-length sed excerpt interleaved stdout writes; the second capture filters these out and shows the complete six-call file sequence.

Execution captures and code excerpts

Native Terminal screenshots: L9 and final L10 were run live; the two labeled audit figures show saved verification output.

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab9-live-jJxiX0# sed -n '/openat.*"note.txt".*O_WRONLY/,$p' syscalls.tra
ce | grep -v '^write(1,'
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(3, "hello, kernel\n", 14)         = 14
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
read(3, "hello, kernel\n", 63)          = 14
close(3)                                = 0
+++ exited with 0 +++
root@ubuntu-4gb-hel1-1:/tmp/igor-lab9-live-jJxiX0# LC_ALL=C strace -o ls.trace -e trace=openat,read,write,
close ls -l > /dev/null
root@ubuntu-4gb-hel1-1:/tmp/igor-lab9-live-jJxiX0# grep '^write(1,' ls.trace
write(1, "total 32\n-rw-r--r-- 1 root root "..., 272) = 272
root@ubuntu-4gb-hel1-1:/tmp/igor-lab9-live-jJxiX0# LC_ALL=C strace -c -e trace=openat ls -l > /dev/null
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
100.00    0.000193          19        10           openat
------ ----------- ----------- --------- --------- ----------------
100.00    0.000193          19        10           total
root@ubuntu-4gb-hel1-1:/tmp/igor-lab9-live-jJxiX0#
```

Figure 2: Native Linux strace: six file calls, ls stdout and openat summary

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab7-live-NLIEGH# lab9_capture_dir=$(mktemp -d /tmp/igor-lab9-live-XXXXXX
)
root@ubuntu-4gb-hel1-1:/tmp/igor-lab7-live-NLIEGH# cp /tmp/igor-os-review-20261109/Lab9-10/syscalls.c "$la
b9_capture_dir/"
root@ubuntu-4gb-hel1-1:/tmp/igor-lab7-live-NLIEGH# cd "$lab9_capture_dir"
root@ubuntu-4gb-hel1-1:/tmp/igor-lab9-live-jJxiX0# gcc -std=c11 -Wall -Wextra -o syscalls syscalls.c
root@ubuntu-4gb-hel1-1:/tmp/igor-lab9-live-jJxiX0# ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
root@ubuntu-4gb-hel1-1:/tmp/igor-lab9-live-jJxiX0# strace -o syscalls.trace -e trace=openat,read,write,clo
se ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
root@ubuntu-4gb-hel1-1:/tmp/igor-lab9-live-jJxiX0# sed -n '/openat.*"note.txt".*O_WRONLY/,+5p' syscalls.tr
ace
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(1, "open returned descriptor 3\n", 27) = 27
write(3, "hello, kernel\n", 14)         = 14
write(1, "write sent 14 bytes\n", 20)   = 20
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
root@ubuntu-4gb-hel1-1:/tmp/igor-lab9-live-jJxiX0# grep '^write(1,' syscalls.trace
write(1, "open returned descriptor 3\n", 27) = 27
write(1, "write sent 14 bytes\n", 20)   = 20
write(1, "read got 14 bytes: hello, kernel"..., 33) = 33
root@ubuntu-4gb-hel1-1:/tmp/igor-lab9-live-jJxiX0#
```

Figure 3: Native Linux strace: direct-terminal stdout writes

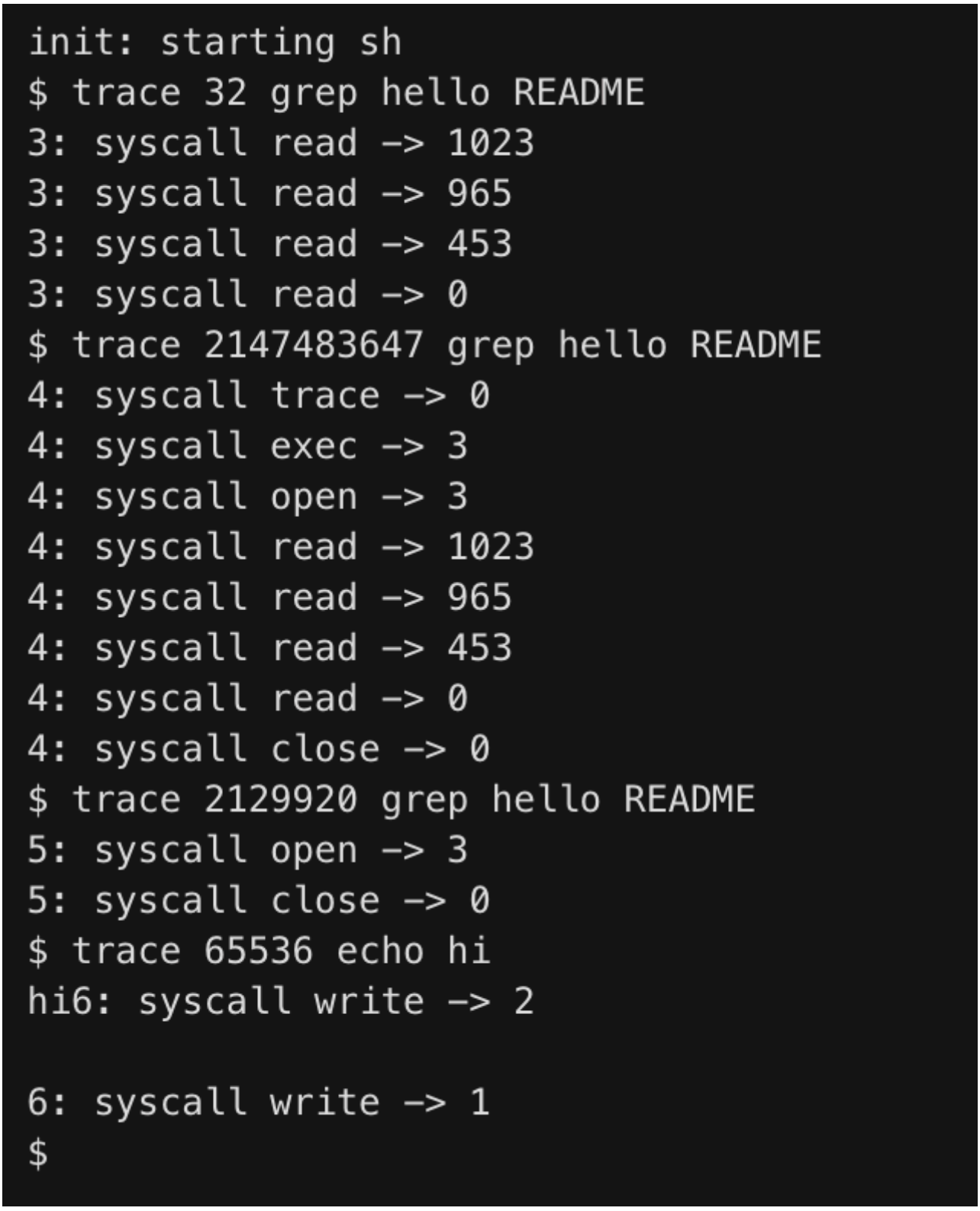


Figure 4: Native xv6 execution: read, all-call, open/close and write masks

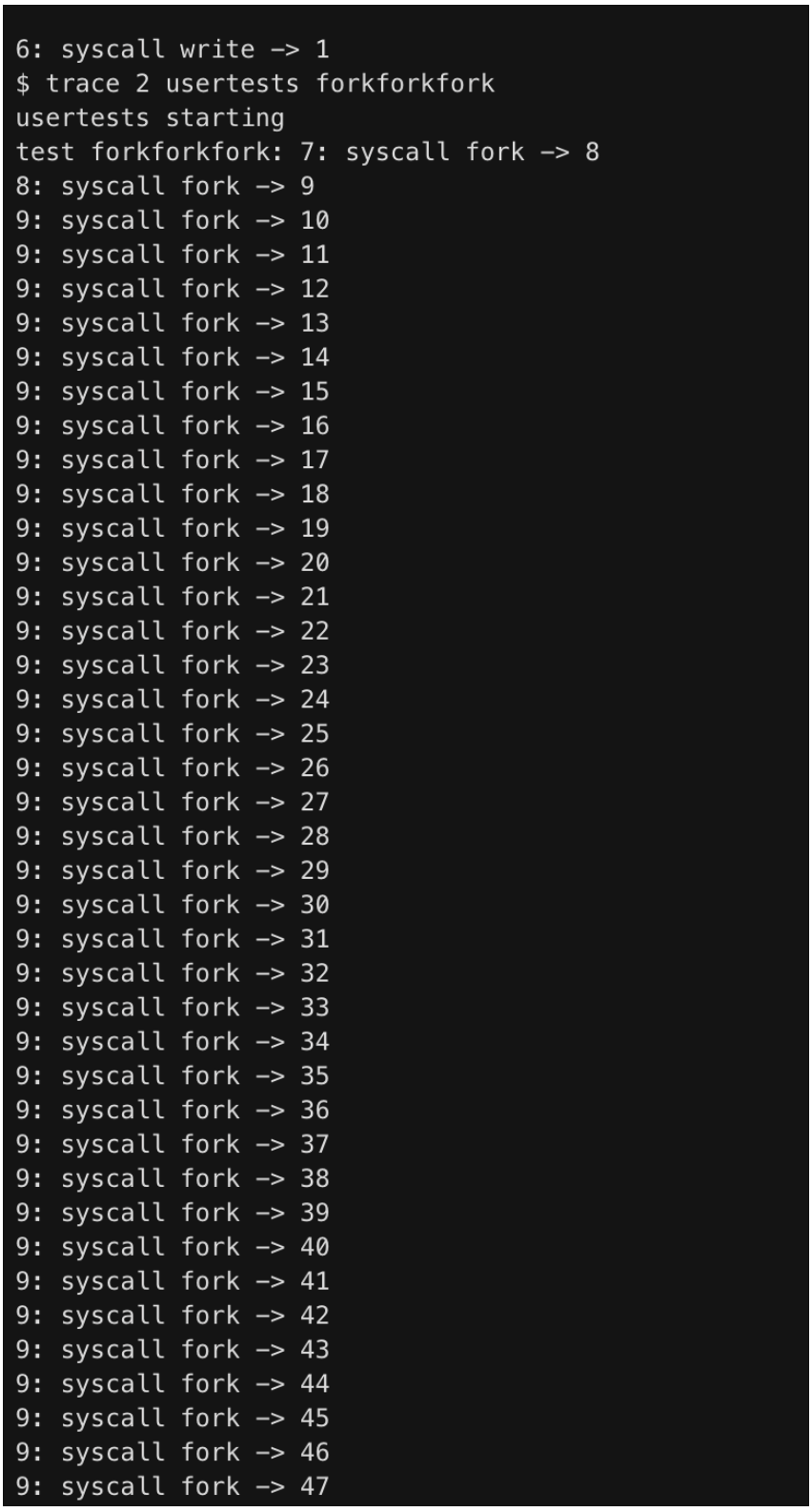


Figure 5: Native xv6 execution: fork test command and inherited tracing

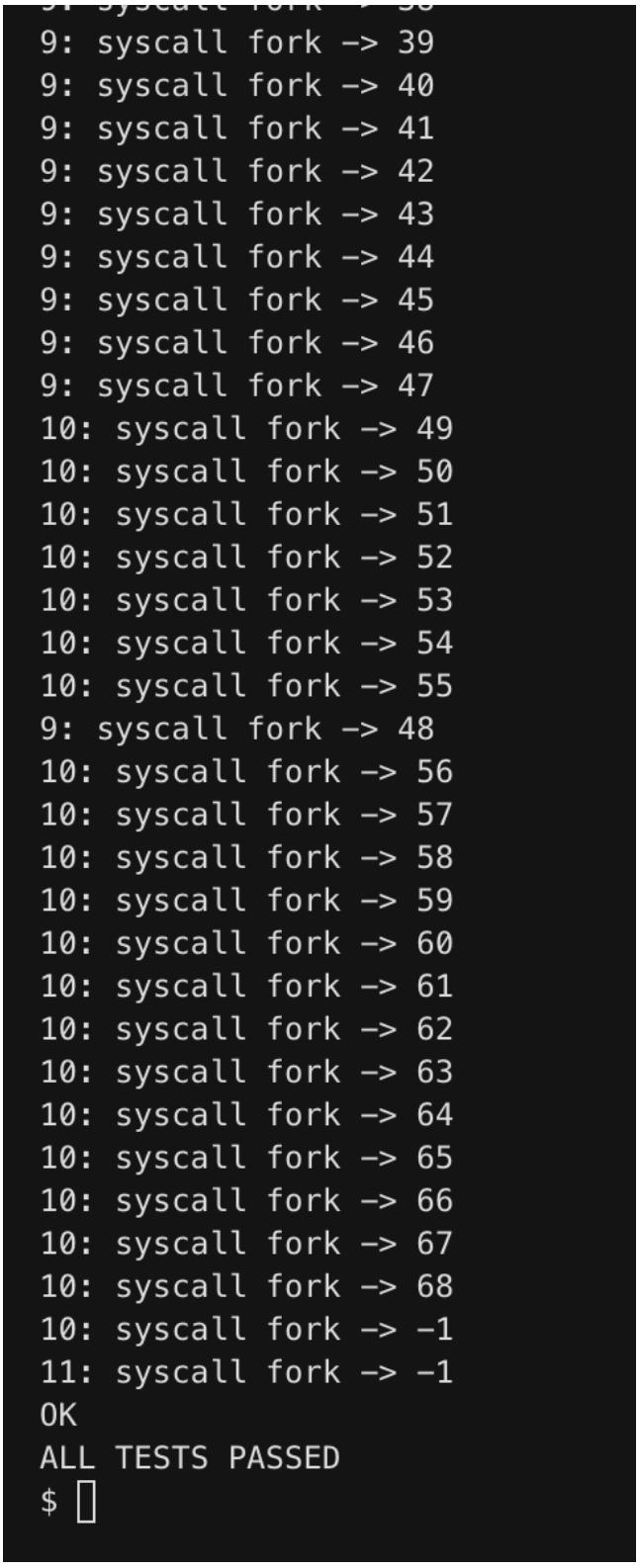


Figure 6: Native xv6 execution: fork test completion and ALL TESTS PASSED

Source excerpts from the submitted implementation

Linux file operations

```
/* Lab 9: observe open, write, read, and close with strace. */
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

int main(void)
{
    const char *msg = "hello, kernel\n";
    int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) {
        perror("open for writing");
        return 1;
    }

    printf("open returned descriptor %d\n", fd);
    ssize_t n = write(fd, msg, strlen(msg));
    if (n < 0 || (size_t)n != strlen(msg)) {
        perror("write");
        close(fd);
        return 1;
    }
    printf("write sent %zd bytes\n", n);
    if (close(fd) < 0) {
        perror("close after writing");
        return 1;
    }

    fd = open("note.txt", O_RDONLY);
    if (fd < 0) {
        perror("open for reading");
        return 1;
    }
    char buf[64];
    n = read(fd, buf, sizeof(buf) - 1);
    if (n < 0) {
        perror("read");
        close(fd);
        return 1;
    }
    buf[n] = '\0';
    printf("read got %zd bytes: %s", n, buf);
    if (close(fd) < 0) {
        perror("close after reading");
        return 1;
    }
    return 0;
}
```

xv6 user utility

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2, "usage: trace mask command [args...]\n");
    exit(1);
  }
  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
  }
  exec(argv[2], argv + 2);
  fprintf(2, "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

Kernel handler

```
uint64
sys_trace(void)
{
  int mask;
  argint(0, &mask);
  myproc()->trace_mask = mask;
  return 0;
}
```

Dispatch after the system call returns

```
void
syscall(void)
{
  int num;
  struct proc *p = myproc();

  num = p->trapframe->a7;
  if (num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    // Use num to lookup the system call function for num, call it,
    // and store its return value in p->trapframe->a0
    p->trapframe->a0 = syscalls[num]();
    if (num < 32 && (p->trace_mask & (1U << num)))
      printk("%d: syscall %s -> %ld\n", p->pid, syscallnames[num], p->trapframe->a0);
  } else {
    printk("%d %s: unknown sys call %d\n", p->pid, p->name, num);
    p->trapframe->a0 = -1;
  }
}
```

Per-process state, initialization and inheritance

These are excerpts from kernel/proc.h and kernel/proc.c respectively; the complete context is in the buildable tree and the validated patch.

```
  struct file *ofile[NOFILE];  // Open files
  struct inode *cwd;           // Current directory
  char name[16];               // Process name (debugging)
  int trace_mask;              // System calls selected for tracing
};


found:
  p->pid = allocpid();
  p->state = USED;
  p->trace_mask = 0;

  // Allocate a trapframe page.
  if ((p->trapframe = (struct trapframe *)kalloc()) == 0) {

    if (p->ofile[i])
      np->ofile[i] = filedup(p->ofile[i]);
  np->cwd = idup(p->cwd);

  safestrcpy(np->name, p->name, sizeof(p->name));
  np->trace_mask = p->trace_mask;

  pid = np->pid;
```

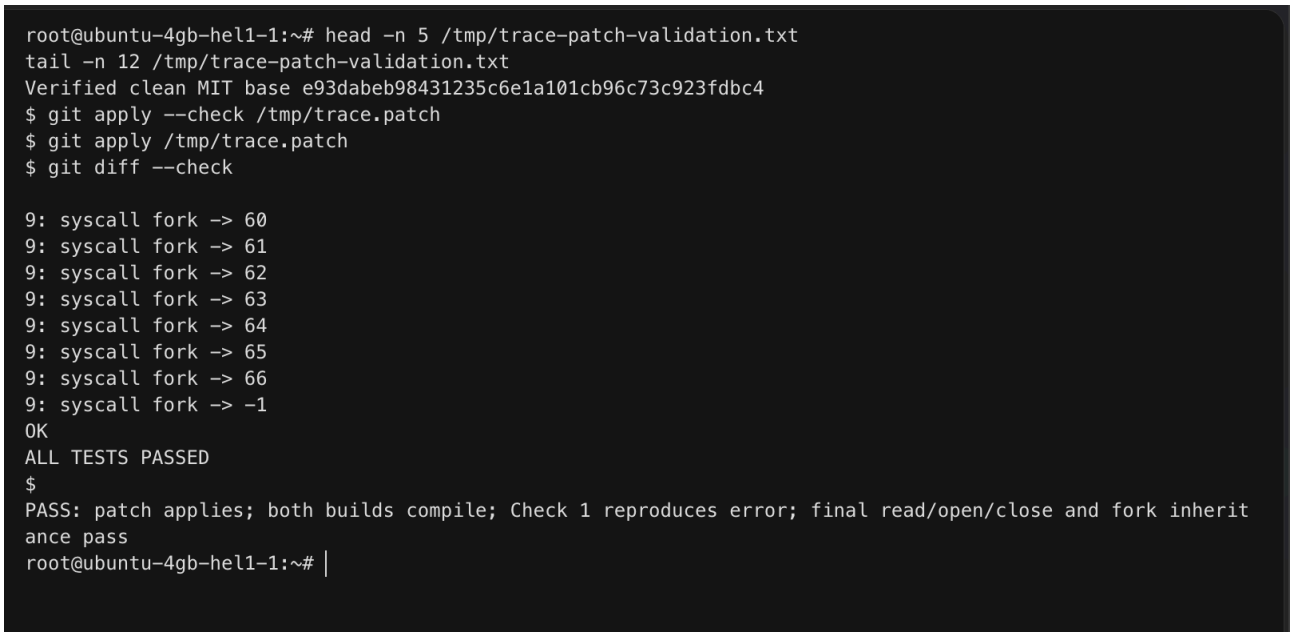


Figure 7: Native Terminal screenshot of the saved audit record: verified MIT base, patch application checks, and passing fork inheritance test. The head and tail commands display the beginning and end of the complete committed log.